Generics, Monaden & Collections

Generics, Monaden & Collections
Gliederung

- 1. Generics vertiefen
- 2. Listen & Collections
- 3. map, flatMap & for-Comprehensions
- 4. Monaden
- 5. Option, Either & Co.
- 6. Übung

innFactory

Generics vertiefen

Generics vertiefen
Rückblick: Typparameter & Varianz

- **VL02:** Generics ermöglichen typparameterisierten, wiederverwendbaren Code
- **Varianz** bestimmt Subtyp-Verhältnis generischer Typen:
  - **Invariant** [A]: Box[Cat] ist **nicht** Box[Animal] (Standard)
  - **Kovariant** [+A]: Producer[Cat] **ist** Producer[Animal] (Subtyp bleibt erhalten)
  - **Kontravariant** [-A]: Consumer[Animal] **ist** Consumer[Cat] (umgekehrt)
- **Faustregel:** Produzenten sind kovariant, Konsumenten sind kontravariant

Generics vertiefen
Invarianz [A]: Lesen und Schreiben

- **Invariant** = Standard in Scala. Kein Subtyp-Verhältnis.
- Array[Cat] ist **nicht** Array[Animal], obwohl Cat <: Animal
- **Warum?** Weil man in ein Array **lesen und schreiben** kann
- Sonst könnte man über einen Animal-Zugriff einen Dog hineinschreiben

```
trait Animal {
  def name: String
}
class Cat(val name: String) extends Animal
class Dog(val name: String) extends Animal

val cats: Array[Cat] = Array(Cat("Minka"), Cat("Felix"))

// val animals: Array[Animal] = cats   // Fehler! Array ist invariant
// Wäre das erlaubt: animals(0) = Dog("Rex")
```

Generics vertiefen
Kovarianz [+A]: Nur Lesen / Produzieren

- **Kovariant** = Subtyp-Beziehung bleibt erhalten
- List[Cat] **ist** ein List[Animal], weil List nur Werte **liefert** (read-only)
- Funktioniert, weil man keine Elemente in eine List hinein-mutieren kann

```
trait Animal { def name: String }
class Cat(val name: String) extends Animal
class Dog(val name: String) extends Animal

val cats: List[Cat] = List(Cat("Minka"), Cat("Felix"))
val animals: List[Animal] = cats   // OK! List ist kovariant [+A]

animals.head.name                  // → "Minka" (sicher, Cat ist Animal)

// Kein Problem: List ist immutable, man kann keinen Dog hineinschreiben
// animals(0) = Dog("Rex")         // gibt es nicht bei List!
```

Generics vertiefen
Kontravarianz [-A]: Nur Schreiben / Konsumieren

- **Kontravariant** = Subtyp-Beziehung **dreht sich um**
- Ein Drucker für alle Materialien kann auch Plastik drucken
- Printer[Material] **ist** ein Printer[Plastik] (umgekehrt wie erwartet!)

```
trait Material {
  def name: String
}
class Plastik(val name: String) extends Material

trait Printer[-A] {              // Kontravariant!
  def print(a: A): String
}
```

Generics vertiefen
Kontravarianz: Printer-Beispiel

- Ein `Printer[Material]` kann überall dort verwendet werden, wo ein `Printer[Plastik]` erwartet wird
- Grund: Er kann **mindestens** Plastik verarbeiten

```
trait Material { def name: String }
class Plastik(val name: String) extends Material

trait Printer[-A] {
  def print(a: A): String
}

val materialPrinter: Printer[Material] =(a: Material) => s“Material: ${a.name}"

val plastikPrinter: Printer[Plastik] = materialPrinter // OK! Material-Printer kann Plastik drucken
plastikPrinter.print(Plastik(“PLA"))   // → “Material: PLA" (sicher!)
```

Generics vertiefen
Rückblick: Bounds

- **VL02:** Upper Bound [A <: T] schränkt ein, Lower Bound [B >: A] erweitert
- **Zusammenspiel mit Varianz:** Kovarianter Typ [+A] braucht Lower Bound für Eingabe-Parameter
- **Warum?** Ohne Lower Bound könnte man in eine List[Cat] einen Dog einfügen

```
// Kovariante Datenstruktur + Lower Bound
sealed trait MyList[+A] {
  def prepend[B >: A](elem: B): MyList[B] =
    Cons(elem, this)
}

val cats: MyList[Cat] = Cons(cat, Empty)
val animals: MyList[Animal] = cats.prepend(dog)  // Typ wird zu MyList[Animal]
```

Generics vertiefen
Typgrenzen kombiniert

- **Upper + Lower Bound** zusammen: [B >: Cat <: Animal]
- **Context Bounds** [A : Ordering]: A braucht eine Ordering-Instanz

```
// Upper + Lower Bound
def clamp[B >: Cat <: Animal](b: B): Animal = b

// Context Bound (Vorschau auf Typklassen)
def sort[A : Ordering](xs: List[A]): List[A] = xs.sorted
sort(List(3, 1, 2))          // → List(1, 2, 3)
sort(List("c", "a", "b"))    // → List("a", "b", "c")
```

Generics vertiefen
Zusammenfassung Generics

- **Typparameter** [A] → generischer, wiederverwendbarer Code
- **Varianz** [+A/-A] → Subtyp-Beziehungen bei generischen Typen
- **Bounds** [<:/>:] → Einschränkungen auf Typparameter
- **Context Bounds** [A : TC] → Typklassen-Anforderungen
- Zusammen ermöglichen sie **typsichere, flexible** APIs

Listen & Collections

Listen & Collections
Wichtige List-Operationen

- **Zugriff:** head, tail, isEmpty, apply(index)
- **Hinzufügen:** :: (vorne, O(1)), :+ (hinten, O(n))
- **Zusammenfügen:** ++ oder ::: (zwei Listen verbinden)
- **Transformieren:** map, filter, flatMap, collect (→ nächstes Kapitel)
- **Falten:** foldLeft, foldRight, reduce (Liste zu einem Wert zusammenfalten)
- **Suchen:** find, exists, forall, contains

```
val xs = List(1, 2, 3, 4, 5)
xs.filter(_ > 2)          // → List(3, 4, 5)
xs.take(3)                // → List(1, 2, 3)
xs.zip(List("a", "b"))    // → List((1,"a"), (2,"b"))
xs.foldLeft(0)(_ + _)     // → 15 (Summe)
xs.exists(_ > 4)          // → true
```

Listen & Collections
Tupel

- **Tupel:** Feste Anzahl von Werten unterschiedlichen Typs bündeln
- Syntax: (1, "Hallo", true), Typ: (Int, String, Boolean)
- Zugriff über ._1, ._2 oder besser via **Pattern Matching**

```
val pair = ("Alice", 30)
pair._1                      // → "Alice"
pair._2                      // → 30
val (name, age) = pair       // Pattern Matching (bevorzugt)
```

Listen & Collections
Map

- **Map[K, V]:** Schlüssel-Wert-Paare, Entries sind Tupel
- "a" -> 1 ist Kurzform für ("a", 1)
- Zugriff mit apply wirft Exception, get gibt **Option** zurück

```
val m = Map("x" -> 1, "y" -> 2)
m("x")                          // → 1
m.get("z")                      // → None (Option!)
m + ("z" -> 3)                  // → Map("x" -> 1, "y" -> 2, "z" -> 3)
m.contains("x")                 // → true
```

Listen & Collections

Collection-Wahl: List vs. Vector

- **List:** Einfach verkettet, O(1) am Kopf, O(n) am Ende und bei Index-Zugriff
  - Ideal für rekursive Verarbeitung (head :: tail)

- **Vector:** Breiter Baum (Faktor 32), O(~1) für Index, append und prepend
  - Ideal als allgemeine Sequenz, wenn Zugriffsmuster unklar

```
val xs = List(1, 2, 3)
xs.head                 // O(1)
xs(2)                   // O(n) - muss durchlaufen!

val vs = Vector(1, 2, 3)
vs(2)                   // O(~1) - effektiver Indexzugriff
vs :+ 4                 // O(~1) - am Ende anfügen
```

Listen & Collections

Collection-Wahl: Array, Set & Map

- **Array:** JVM-Primitiv, O(1) Index, aber **mutable** und fixe Größe
- **Set:** O(~1) Zugriff (Hash-basiert), keine Duplikate
- **Map:** O(~1) Zugriff (Hash-basiert), Schlüssel-Wert-Paare
- **Faustregel:** Im Zweifel Vector statt List, wenn keine rekursive Zerlegung nötig

```
val arr = Array(1, 2, 3)
arr(0)                    // O(1)
arr(0) = 99               // Mutation! Einzige mutable Collection

val s = Set(1, 2, 2, 3)   // → Set(1, 2, 3) - keine Duplikate
s.contains(2)             // O(~1)
```

Listen & Collections
groupBy & partition

- **groupBy:** Elemente nach Kriterium gruppieren → Map[K, Collection[V]]
- **partition:** In zwei Gruppen teilen (erfüllt / erfüllt nicht)

```
val words = List("Scala", "ist", "super", "stark")
words.groupBy(_.length)
// → Map(5 -> List("Scala", "super", "stark"), 3 -> List("ist"))
words.partition(_.length > 3)
// → (List("Scala", "super", "stark"), List("ist"))
```

Listen & Collections
collect & scan

- **collect:** map + filter kombiniert mit PartialFunction
- **scanLeft:** fold mit allen Zwischenergebnissen

```
List(1, -2, 3, -4).collect { case x if x > 0 => x * 10 }
// → List(10, 30)
List(1, 2, 3, 4).scanLeft(0)(_ + _)
// → List(0, 1, 3, 6, 10)
List(1, 2, 3).sliding(2).toList
// → List(List(1, 2), List(2, 3))
```

map, flatMap & for-Comprehensions

map, flatMap & for-Comprehensions
map: Transformation

- **map** wendet eine Funktion auf **jeden Wert im Container** an
- Der Container-Typ bleibt gleich, der Inhalt wird transformiert
- Funktioniert auf List, Option, Either, Future, ... (überall gleich!)
- **Signatur:** def map[B](f: A => B): F[B]

```
// List: Jeden Wert transformieren
List(1, 2, 3).map(x => x * 2)       // → List(2, 4, 6)
List("a", "b").map(_.toUpperCase)     // → List("A", "B")
// Option: Wert transformieren, wenn vorhanden
Some(5).map(_ * 2)          // → Some(10)
None.map((x: Int) => x * 2) // → None (Funktion wird nicht aufgerufen)
// Either: Rechten Wert transformieren
Right(42).map(_ + 1)        // → Right(43)
Left("Fehler").map((x: Int) => x + 1)  // → Left("Fehler")
```

map, flatMap & for-Comprehensions
flatMap: Transformation + Flachklopfen

- **flatMap** = map + flatten: Funktion liefert Container, der "ausgepackt" wird
- Verhindert **verschachtelte Container** wie Option[Option[Int]]
- **Signatur:** def flatMap[B](f: A => F[B]): F[B]

```
List(1, 2, 3).flatMap(x => List(x, x * 10))
// → List(1, 10, 2, 20, 3, 30)
List("Hallo Welt", "Hi").flatMap(_.split(" ").toList)
// → List("Hallo", "Welt", "Hi")
```

map, flatMap & for-Comprehensions
flatMap: Verkettung mit Option

- Entscheidend für **Verkettung** von Operationen, die fehlschlagen können
- Bei None bricht die Kette sofort ab

```
def findUser(id: Int): Option[String] =
  if (id == 1) Some("Alice") else None
def findEmail(name: String): Option[String] =
  if (name == "Alice") Some("alice@example.com") else None

findUser(1).flatMap(findEmail)   // → Some("alice@example.com")
findUser(99).flatMap(findEmail)  // → None (Kette bricht ab)
```

map, flatMap & for-Comprehensions
pure: Wert in den Kontext heben

- **pure** packt einen normalen Wert in einen Kontext/Container
- Signatur: `A => F[A]`
- Beispiele: `Some(42)`, `Right(42)`, `List(42)`

```
// Werte in den Kontext heben
Some(42)     // Option[Int]
Right(42)    // Either[E, Int]
List(42)     // List[Int]
```

map, flatMap & for-Comprehensions

for-Comprehensions: Syntaktischer Zucker

- **for-yield** ist syntaktischer Zucker für map/flatMap-Ketten
- Liest sich wie imperativer Code, ist aber funktional (keine Mutation!)
- Jedes <- ist ein flatMap (außer das letzte, das ist map)
- Funktioniert überall, wo map und flatMap definiert sind

```
// Verschachtelte flatMaps sind schwer lesbar:
findUser(1).flatMap(name =>
  findEmail(name).map(email => s"$name: $email")
)
```

map, flatMap & for-Comprehensions
for-Comprehensions: Äquivalenz

- Gleiche Logik, aber **deutlich lesbarer** als verschachtelte flatMaps
- Compiler übersetzt for/yield automatisch in flatMap/map-Aufrufe

```
// Eleganter mit for-Comprehension:
for {
  name  <- findUser(1)
  email <- findEmail(name)
} yield s"$name: $email"
// → Some("Alice: alice@example.com")

// Compiler erzeugt daraus:
findUser(1).flatMap(name =>
  findEmail(name).map(email => s"$name: $email")
)
```

map, flatMap & for-Comprehensions
for-Comprehensions auf Listen

- Auf Listen: Jedes <- erzeugt eine **geschachtelte Schleife**
- Ergebnis: Alle Kombinationen (Kreuzprodukt)
- Gleiche Syntax wie bei Option, aber andere Semantik

```
for {
  x <- List(1, 2, 3)
  y <- List("a", "b")
} yield s"$x$y"
// → List("1a", "1b", "2a", "2b", "3a", "3b")
```

map, flatMap & for-Comprehensions
for-Comprehensions: Guards

- **Guards** (if): Werte filtern, entspricht withFilter
- Mehrere Guards und Generatoren kombinierbar

```
// Guards: Nur gerade Zahlen
for {
  x <- List(1, 2, 3, 4, 5)
  if x % 2 == 0
} yield x * 10
// → List(20, 40)
```

map, flatMap & for-Comprehensions
for-Comprehensions: Zuweisungen

- **Zuweisungen** (=): Zwischenergebnisse benennen, kein flatMap nötig
- Destrukturierung von Tupeln direkt in der for-Comprehension
- Kompiliert zu einer Kombination aus flatMap, map und withFilter

```
// Zuweisungen: Zwischenergebnisse
for {
  pair <- Map("Alice" -> 30, "Bob" -> 25)
  (name, age) = pair           // Tupel-Destructuring
  if age >= 30
} yield s"$name ist $age"
// → List("Alice ist 30")
```

map, flatMap & for-Comprehensions
for-Comprehensions in der Praxis

- **Typisches Muster:** Mehrere Schritte, die jeweils fehlschlagen können
- Jeder Schritt gibt Option oder Either zurück, for verkettet automatisch
- Bei **None** oder **Left:** Restliche Schritte werden übersprungen

```
case class User(name: String, age: Int)
case class Ticket(user: User, seat: String)

def findUser(id: Int): Option[User] =
  Map(1 -> User("Alice", 30), 2 -> User("Bob", 17)).get(id)
def checkAge(user: User): Option[User] =
  if (user.age >= 18) Some(user) else None
def assignSeat(user: User): Option[Ticket] =
  Some(Ticket(user, "A12"))
```

map, flatMap & for-Comprehensions
for-Comprehensions: Verkettung

- Liest sich wie sequentieller Code, ist aber rein funktional
- Bei Fehler (None) bricht die Kette sofort ab

```
// Erfolgsfall: Alice, 30 Jahre
val booking: Option[Ticket] = for {
  user    <- findUser(1)
  checked <- checkAge(user)
  ticket  <- assignSeat(checked)
} yield ticket
// → Some(Ticket(User("Alice", 30), "A12"))
```

map, flatMap & for-Comprehensions
for-Comprehensions: Fehlerfall

- checkAge gibt None für minderjährige User → Kette bricht ab
- assignSeat wird **nie aufgerufen**, Rest der Kette wird übersprungen

```
// Fehlerfall: Bob, 17 Jahre
val fail: Option[Ticket] = for {
  user    <- findUser(2)
  checked <- checkAge(user)      // None!
  ticket  <- assignSeat(checked) // wird übersprungen
} yield ticket
// → None
// findUser(99) → None: Schon der erste Schritt schlägt fehl
```

map, flatMap & for-Comprehensions
Funktor: Die Abstraktion hinter map

- **Beobachtung:** map funktioniert auf List, Option, Either, Future, ... immer gleich
- Ein **Funktor** ist alles, was eine sinnvolle map-Operation hat
- **Formal:** Ein Typ F[_] mit map[A, B](fa: F[A])(f: A => B): F[B]

```
// Alle diese Typen sind Funktoren:
List(1, 2, 3).map(_ + 1)       // → List(2, 3, 4)
Some(5).map(_ + 1)              // → Some(6)
Right(5).map(_ + 1)             // → Right(6)
```

map, flatMap & for-Comprehensions
Funktor-Gesetze

- **Identität:** fa.map(x => x) == fa
- **Komposition:** fa.map(f).map(g) == fa.map(x => g(f(x)))
- Gesetze garantieren, dass map den Container nicht "verfälscht"

```
// Identitätsgesetz
List(1, 2, 3).map(x => x)      // → List(1, 2, 3) ✓
Some(42).map(x => x)            // → Some(42) ✓
// Kompositionsgesetz
List(1, 2).map(_ + 1).map(_ * 2)          // → List(4, 6)
List(1, 2).map(x => (x + 1) * 2)          // → List(4, 6) ✓
```

map, flatMap & for-Comprehensions
Vom Funktor zur Monade

- **Funktor:** Hat map → Werte im Container transformieren
- **Monade:** Hat map + flatMap + pure → Berechnungen **verketten**
- **Das Problem ohne flatMap:** map erzeugt verschachtelte Container
- **Hierarchie:** Jede Monade ist ein Funktor, aber nicht jeder Funktor ist eine Monade
- **Warum wichtig?** flatMap entscheidet, ob und wie die Kette weitergeht

```
// Nur mit map: Verschachtelung!
Some(42).map(x => if (x > 0) Some(x) else None)
// → Some(Some(42)) ← Option[Option[Int]], unbrauchbar!

// Mit flatMap: Flach!
Some(42).flatMap(x => if (x > 0) Some(x) else None)
// → Some(42) ← Option[Int], genau richtig!
```

Monaden

Monaden
Was ist eine Monade? (Intuitiv)

- Eine Monade ist ein **Designpattern** für die Verkettung von Berechnungen
- Jede Monade ist ein **Container/Kontext**, der einen Wert einpackt
- Zwei Operationen machen den Container zur Monade:
  - **pure/unit:** Wert einpacken → Some(42), Right(42), List(42)
  - **flatMap:** Verkettete Berechnung, die den Kontext beibehält
- **Intuition:** Monaden sind "programmierbare Semikolons"
  - Jeder Schritt entscheidet, ob und wie es weitergeht
  - Option: Weiter nur bei Some, stopp bei None
  - Either: Weiter nur bei Right, stopp bei Left
  - List: Jeder Wert erzeugt mehrere Nachfolger

Monaden
Was ist eine Monade? (Definition)

- Eine Monade ist ein **Typkonstruktor** `F[_]` mit zwei zentralen Operationen:
  - `pure: A => F[A]` packt einen Wert in den Kontext
  - `flatMap: F[A] => (A => F[B]) => F[B]` verkettet Berechnungen im Kontext
- `map` ist daraus ableitbar: `fa.map(f) == fa.flatMap(a => pure(f(a)))`
- Eine Monade muss außerdem die **drei Monaden-Gesetze** erfüllen

```
// Praktisch in Scala denkt man so:
// 1. Wert in Kontext packen
Some(42)

// 2. Berechnungen im selben Kontext verketten
Some(42).flatMap(x => Some(x + 1))   // → Some(43)
```

Monaden

Monaden, die wir schon kennen

- **Option[A]:** Kontext "Wert vorhanden oder nicht" (pure: Some(x))
- **Either[L, R]:** Kontext "Erfolg oder Fehler" (pure: Right(x))
- **List[A]:** Kontext "Mehrere mögliche Werte" (pure: List(x))
- Alle drei unterstützen **for-Comprehensions**, weil sie map + flatMap haben

```
// Dasselbe Pattern, drei verschiedene Monaden:
for { a <- Some(1); b <- Some(2) } yield a + b         // → Some(3)
for { a <- Right(1); b <- Right(2) } yield a + b       // → Right(3)
for { a <- List(1, 2); b <- List(10, 20) } yield a + b // → List(11, 21, 12, 22)
```

Monaden
Die drei Monaden-Gesetze

- Jede "echte" Monade muss drei Gesetze erfüllen:
- **1. Left Identity:** pure(a).flatMap(f) == f(a)
- **2. Right Identity:** m.flatMap(pure) == m
- **3. Assoziativität:** m.flatMap(f).flatMap(g) == m.flatMap(x => f(x).flatMap(g))

```scala
val f: Int => Option[Int] = x => Some(x + 1)
val g: Int => Option[Int] = x => Some(x * 2)
// Left Identity
Some(5).flatMap(f) == f(5)                        // Some(6) == Some(6) ✓
// Right Identity
Some(5).flatMap(Some(_)) == Some(5)               // true ✓
// Assoziativität
Some(5).flatMap(f).flatMap(g)                     // → Some(12)
Some(5).flatMap(x => f(x).flatMap(g))             // → Some(12) ✓
```

Monaden
Warum Monaden-Gesetze?

- Gesetze garantieren **vorhersagbares Verhalten** bei Refactoring
- Ohne Gesetze: Code-Umstellungen könnten Ergebnisse ändern
- **Assoziativität** erlaubt das Umklammern von for-Comprehensions
- **Identity-Gesetze** sichern, dass pure/Some/Right/List "neutral" sind
- **Bezug zum Lambda-Kalkül:** Monaden-Gesetze sind Äquivalenzen wie β-Reduktion
  - Left Identity ≈ β-Reduktion: $(\lambda x.\ f(x))(a) = f(a)$

Option, Either & Co.

Option, Either & Co.
Option vertieft

- **Option[A]** = Some(wert) | None
- Ersetzt **null** komplett: Der Typ zeigt explizit, dass ein Wert fehlen kann
- **map/flatMap:** Wert transformieren bzw. verketten
- **getOrElse/orElse:** Standardwert oder Alternative bei None

```
val opt: Option[Int] = Some(42)
opt.map(_ * 2)              // → Some(84)
opt.flatMap(x => if (x > 0) Some(x) else None)  // → Some(42)
opt.getOrElse(0)            // → 42
None.getOrElse(0)           // → 0
opt.orElse(Some(0))         // → Some(42)
```

Option, Either & Co.
Option: fold & Pattern Matching

- **fold:** Beide Fälle (Some/None) explizit in einem Aufruf behandeln
- **Pattern Matching:** Alternativer, ausführlicherer Stil

```
val opt: Option[Int] = Some(42)
opt.fold("leer")(x => s"Wert: $x")  // → "Wert: 42"
None.fold("leer")((x: Int) => s"Wert: $x")  // → "leer"

opt match {
  case Some(v) => s"Gefunden: $v"
  case None    => "Nicht gefunden"
}
// → "Gefunden: 42"
```

Option, Either & Co.
Either vertieft

- **Either[L, R]** = Left(fehler) | Right(erfolg)
- Konvention: Left = Fehler, Right = Erfolg (right = richtig)
- Either ist **right-biased:** map/flatMap arbeiten auf dem Right-Wert
- Ersetzt Exceptions für **erwartete Fehler** (Validierung, Parsing, IO)

```
def parseInt(s: String): Either[String, Int] =
  try Right(s.toInt) catch {
    case _: NumberFormatException => Left(s"'$s' ist keine Zahl")
  }
def checkPositive(n: Int): Either[String, Int] =
  if (n > 0) Right(n) else Left("Muss positiv sein")

parseInt("42").flatMap(checkPositive).map(_ * 2)  // → Right(84)
parseInt("abc").flatMap(checkPositive)  // → Left("'abc' ist keine Zahl")
```

Option, Either & Co.
Either: Eigene Fehlertypen

- **Eigene Fehlertypen** statt Strings: Typsichere Fehlerbeschreibung mit ADTs
- Compiler kann **Vollständigkeit** prüfen bei Pattern Matching auf Fehlertypen

```
enum AppError {
  case NotFound(id: Int)
  case InvalidInput(msg: String)
  case Unauthorized
}

def findUser(id: Int): Either[AppError, User] = ...
def checkPermission(user: User): Either[AppError, User] = ...
def loadProfile(user: User): Either[AppError, Profile] = ...
```

Option, Either & Co.
Either: Fehlerketten mit for

- **Mehrere Fehlerquellen** elegant mit for-Comprehension verketten
- Erster Left-Wert bricht die Kette ab

```
val result: Either[AppError, Profile] =
  for {
    user    <- findUser(42)
    allowed <- checkPermission(user)
    profile <- loadProfile(allowed)
  } yield profile
```

Option, Either & Co.
Either: Fehlerbehandlung

- Am Ende: **Ein** Pattern Match für alle möglichen Fehler
- Compiler prüft Vollständigkeit bei sealed Fehlertypen

```
result match {
  case Right(profile) =>
    showProfile(profile)
  case Left(AppError.NotFound(id)) =>
    show404(id)
  case Left(AppError.InvalidInput(m)) =>
    showError(m)
  case Left(AppError.Unauthorized) =>
    showLogin()
}
```

Option, Either & Co.
Praxisbeispiel: Service-Pipeline

- **Reales Szenario:** HTTP-Request verarbeiten mit mehreren Fehlerquellen
- Jeder Schritt kann fehlschlagen, alle Fehler typisiert

```
case class Request(body: String)
case class Order(userId: Int, product: String, qty: Int)

def parseJson(req: Request): Either[AppError, Order] =
  Right(Order(42, "Buch", 2))

def validateOrder(order: Order): Either[AppError, Order] =
  if (order.qty > 0) Right(order)
  else Left(AppError.InvalidInput("Menge muss positiv sein"))
```

Option, Either & Co.
Service-Pipeline: Verkettung

- for-Comprehension hält die Logik **lesbar und linear**
- Der Happy Path bleibt kompakt und lesbar

```
def processOrder(req: Request): Either[AppError, String] = {
  for {
    order   <- parseJson(req)
    valid   <- validateOrder(order)
    user    <- findUser(valid.userId)
    allowed <- checkPermission(user)
  } yield s"Bestellung für ${user.name}: ${valid.qty}x ${valid.product}"
}
```

Option, Either & Co.
Service-Pipeline: Ergebnis auswerten

- Am Ende reicht **ein** Pattern Match für Erfolg und Fehler

```
processOrder(Request("...")) match {
  case Right(msg)  => println(msg)
  case Left(error) => println(s"Fehler: $error")
}
```

Option, Either & Co.
Try: Exceptions abfangen

- **Try[A]** = Success(wert) | Failure(exception)
- Fängt Exceptions ab und verpackt sie in einen Wert (keine Seiteneffekte!)
- Auch eine Monade: map, flatMap, for-Comprehension funktionieren
- **Wann Try statt Either?** Für **unerwartete** Exceptions

```
import scala.util.Try
val ok = Try("42".toInt)      // → Success(42)
val fail = Try("abc".toInt)   // → Failure(NumberFormatException)
ok.map(_ * 2)                 // → Success(84)
for { a <- Try("10".toInt); b <- Try("20".toInt) } yield a + b
ok.toEither                   // → Right(42)
```

Option, Either & Co.
Vergleich: Option vs. Either vs. Try

- Alle drei sind **Monaden** mit map, flatMap, for-Comprehension
- Unterschied: Wie viel Information über den **Fehlerfall** mitgeliefert wird
- **Option:** Some(wert) / None → Wert da oder nicht, Grund egal
- **Either:** Right(wert) / Left(fehler) → Eigene Fehlertypen, Validierung
- **Try:** Success(wert) / Failure(exception) → Exceptions abfangen (IO, Parsing)
- **Konvertierungen:** Option ↔ Either ↔ Try
  - option.toRight("Fehler") → Either
  - either.toOption → Option
  - try_.toEither → Either

Option, Either & Co.
Pattern Matching: Option

- **VL02:** Pattern Matching auf ADTs (enum, sealed trait)
- Jetzt: Pattern Matching auf **Option, Either, Tupel**

- Option kann genauso direkt per Pattern Matching ausgewertet werden

```
def greet(name: Option[String]): String = name match {
  case Some(n) => s"Hallo $n"
  case None    => "Hallo Unbekannter"
}
```

Option, Either & Co.
Pattern Matching auf Monaden

- Either-Pattern: Right(wert), Left(fehler)
- **collect** mit Pattern Matching: map + filter in einem Schritt

```
// Either-Pattern
def handle(result: Either[String, Int]): String = result match {
  case Right(value) => s"Erfolg: $value"
  case Left(error)  => s"Fehler: $error"
}

// Verschachtelt: List von Options
List(Some(1), None, Some(3)).collect { case Some(v) => v }
// → List(1, 3)
```

Übung Box[A] | Eigene Monade implementieren

Übung Box[A] | Eigene Monade implementieren
Teil 1: Box + map

- Einen generischen Wrapper implementieren
- **map** transformiert den Inhalt, der Container bleibt gleich

```
case class Box[A](value: A) {
  def map[B](f: A => B): Box[B] = ???
}

// Tests:
Box(5).map(_ * 2)              // → Box(10)
Box("hallo").map(_.length)     // → Box(5)
```

Übung Box[A] | Eigene Monade implementieren
Teil 2: Box + flatMap

- **flatMap** vermeidet verschachtelte Container
- Funktion gibt bereits eine Box zurück → nicht nochmal einpacken

```
case class Box[A](value: A) {
  def map[B](f: A => B): Box[B] = ???
  def flatMap[B](f: A => Box[B]): Box[B] = ???
}

// Tests, was ist der Unterschied?
Box(5).map(x => Box(x * 2))       // → Box(Box(10)) verschachtelt!
Box(5).flatMap(x => Box(x * 2))   // → Box(10)      flach!
```

Übung Box[A] | Eigene Monade implementieren
Teil 3: for-Comprehension testen

- for-yield funktioniert **automatisch**, wenn map + flatMap vorhanden sind
- Kein Interface nötig, kein Trait implementieren

```
val result = for {
  a <- Box(3)
  b <- Box(4)
} yield a + b   // → Box(7)
```

Übung Box[A] | Eigene Monade implementieren
Teil 4: Monaden-Gesetze prüfen

- **Bonus:** Überprüfe die drei Monaden-Gesetze an konkreten Beispielen
- Left Identity: Box(a).flatMap(f) == f(a)
- Right Identity: m.flatMap(Box(_)) == m
- Assoziativität: m.flatMap(f).flatMap(g) == m.flatMap(x => f(x).flatMap(g))

```
val f: Int => Box[Int] = x => Box(x + 1)
val g: Int => Box[Int] = x => Box(x * 2)
val m = Box(5)

// Prüfe alle drei Gesetze mit konkreten Werten!
```

Übung Box[A] | Eigene Monade implementieren
Teil 5: MaybeBox | Box mit leer-Zustand

- Eigene Option nachbauen als enum mit Full und Empty
- Neue Konzepte: **Kovarianz** [+A], **Lower Bound** [B >: A], **Call-by-Name** => B

```
enum MaybeBox[+A] {
  ???
}
```

Übung Box[A] | Eigene Monade implementieren
Teil 6: Kettenabbruch testen

- flatMap auf Empty gibt sofort Empty zurück → Rest wird übersprungen
- Gleiche Semantik wie bei Option/None

```
import MaybeBox.*
def safeDivide(a: Int, b: Int): MaybeBox[Int] = ???

for {
  a <- Full(20)
  b <- safeDivide(a, 4)   // Full(5)
  c <- safeDivide(b, 0)   // Empty → Rest wird übersprungen
} yield c                 // → Empty
```

Übung Box[A] | Eigene Monade implementieren
Teil 7: withFilter

- withFilter implementieren, damit **if-Guards** in for funktionieren
- Ohne withFilter: Compiler-Fehler bei if in for-Comprehension

```
def withFilter(p: A => Boolean): MaybeBox[A] = ???

for {
  x <- Full(5)
  if x > 3        // ← braucht withFilter!
} yield x * 2       // → Full(10)
```

Übung Box[A] | Eigene Monade implementieren
Teil 8: Praxis-Pipeline

- Alles zusammen in einer Studenten-Pipeline
- for-Comprehension mit eigener Monade, getOrElse am Ende

```
case class Student(name: String, grade: Double)
val students = Map(1 -> Student("Alice", 1.3), 2 -> Student("Bob", 5.0))

def findStudent(id: Int): MaybeBox[Student] = ???
def checkPassed(s: Student): MaybeBox[Student] = ???

for {
  student <- findStudent(1)
  passed  <- checkPassed(student)
} yield s"${passed.name} bestanden mit ${passed.grade}"
```

Übung Box[A] | Eigene Monade implementieren
Teil 9 (Bonus): fold

- **fold** behandelt beide Fälle explizit, Full und Empty
- Zeigt auch nochmal **Currying** (zwei Parameterlisten)

```
def fold[B](ifEmpty: => B)(f: A => B): B = ???

// Tests:
Full(42).fold("leer")(x => s"Wert: $x")      // → "Wert: 42"
Empty.fold("leer")((x: Int) => s"Wert: $x")  // → "leer"
```